

School of Information Technologies

Faculty of Engineering & IT

ASSIGNMENT/PROJECT COVERSHEET - GROUP ASSESSMENT

Unit of Study: COMP4348 COMP5348 Enterprise Scale Software Architecture

Assignment name: Group Project Enterprise-Scale Software Development

Tutorial time: Tuesday 5pm, 2 hours **Tutor name:** Xinyi Sheng

DECLARATION

We the undersigned declare that we have read and understood the *University of Sydney Student Plagiarism: Coursework Policy and Procedure*, and except where specifically acknowledged, the work contained in this assignment/project is our own work, and has not been copied from other sources or been previously submitted for award or assessment.

We understand that failure to comply with the *Student Plagiarism: Coursework Policy and Procedure* can lead to severe penalties as outlined under Chapter 8 of the *University of Sydney By-Law 1999* (as amended). These penalties may be imposed in cases where any significant portion of my submitted work has been copied without proper acknowledgement from other sources, including published works, the internet, existing programs, the work of other students, or work previously submitted for other awards or assessments.

We realise that we may be asked to identify those portions of the work contributed by each of us and required to demonstrate our individual knowledge of the relevant material by answering oral questions or by undertaking supplementary work, either written or in the laboratory, in order to arrive at the final assessment mark.

Project team members				
Student name	Student ID	Participated	Agree to share	Signature
1. Ky Anh Pham	520352045	Yes	Yes	
2. Lixiangcheng Dong	530356589	Yes	Yes	
3. Minghang Li	510368843	Yes	Yes	
4. Haoxiong Hu	510601795	Yes	Yes	

Design:

1. Store Application:

1. **Database:** Keeping track of Delivery, Item, Order, OutboxEvent, Payment and User information in subsequent separated relations. The Store apps also maintain another service in charge of inventory management and warehouses called Warehouse, the relations that this app operates on are: Reservation and WarehouseStock.
2. **Application System:** The system holds separated layers of operations, ensuring a separation of concerns between layers. In the Store apps, there are model, DTO, repository, service and middleware layers, this design is to strictly follow MVC design patterns of standard web application.
 - **Restful APIs:** implemented RestAPIs for Delivery, Item, Order, Payment and User for said resources.
 - Delivery ("/api/deliveries/{delivery_id}"): Support only GET requests for clients to track their deliveries. Allows users to get a single or get all deliveries associated with the current user.
 - Item ("/api/item/{item_id}"): Support only GET requests for clients to retrieve available items, it supports get by the item id or get all items registered in the system.
 - Order ("/api/orders/{order_id}"): Support POST, GET, allows clients to retrieve their order at scale (one or many), create, confirm and cancel their order.
 - Payment("/api/payments/{payment_id}"): Support POST and GET request. Allow clients to retrieve their payment information and let clients create, initiate a refund or callback a payment.
 - User ("api/users/"): Support GET ("/me") to retrieve user own account information
 1. POST: ("/signup", "/login", "/logout): Allow user to create an account, login (request authentication token) and logout (invalidate current authentication token)
 2. PUT: Allow users to update the information of their account.
 3. DELETE: Allow users to delete their account.
 - **MessageQueue service (RabbitMQ):** MQ is set to listen for incoming messages from DeliveryCo, EmailService and BankApp for updates on ongoing operations and state changes in these 3rd party apps.
 - The most usually seen mechanism in this project is the implementation of call-back. This is used in the communication with Bank, DeliveryCo as a way to make these 3rd party apps behave in a desired way so that internal resources of the Store app could keep its state at the most up to date condition.
 - Call-back can be explained as a type of promise protocol made by the receiver of the request. For example, if A sends a call-back request to B, after B receives the request, B would have to send a request with an update on the resource of interest's state back to A.

- **gRPC service:** gRPC operates on warehouse management systems, especially on inventory and warehouse state management. This includes reporting available items when the client wants to create an order and subsequently updating the inventory after the client's order creation was followed through.

2. Bank Application:

1. **Database:** Keep track of bank account information and transaction information as BankAccount and BankTransaction relations. These relations have a series of attributes to aid in the app's query operation. Important state indicator attributes are being represented as enum types to keep data integrity high.
2. **Application systems:** The BankApp is also implemented in a standard Spring MVC design pattern by having model, repository, service and controller layers delivering the main feature and enforcing business logic while communicating with each other using custom DTOs and minimise actual resources exposure (BankAccount and BankTransaction entities never escapes repository layer).
 - **Restful APIs :** has 2 Rest APIs for account and payment resource.
 - Account - `"/api/v1/accounts"`: Allows POST and GET functionalities, letting clients create their bank account and retrieve their account information.
 - Payment - `"/api/v1/payment"`: Allows POST and GET functionalities, letting clients make a payment, retrieve their payment information (transaction reference)
 - **Call-back via webhooks:** The main functionality that lets BankApp reports its resources of interest's state to Store. This is triggered whenever there is a state change in the resource state in BankApp and a dedicated channel from Store is open to listen to these requests.
 - **Utilisation of Idempotency Key:** Set up via order-scoped configurations.

3. DeliveryCo Application:

1. **Database:** Postgres database was used to store delivery instance
2. **Backend system:** Follows MVC structure with 5 layers (model, repository, DTOs, service and controller)
 - This helps satisfy the separation of concern requirements. Each layer has its own responsibility to perform and doesn't handle multiple processes. For example, the controller layer handles incoming requests and sanitising request body's data, repository layer handles the communication between the application and the database, as for the service layer, it abstracts supported operations that defines the business logic of DeliveryCo and these operations are being used by the controller layer.
 - Having an ORM helps with readability and maintainability of the code base. It also helps tremendously in security because of a lower chance of being attacked by SQL Injection.
 - The scheduled task to automatically shift the delivery status lives in the Repository layer of DeliveryInstance. This might seem to be a violation of separation of concern but given that this module's only purpose is to operate on DeliveryInstance objects. It makes sense to put this module in the Repository layer, where the application and database communicate.

3. **APIs design:** Follows RESTful API design, having 1 URI serving CRUD operations on objects. However for the purpose of business logic, the delete capability was not supported.
- /api/rest/v1/delivery/ POST, GET, UPDATE/PATCH
 - POST batch form hitting endpoint /batch:
 - [{"details": str, "type": enums_str, "toAddress": str, "fromAddress": str, "orderId: str}, etc...]
 - The /batch URI supported multiple creation of DeliveryInstance to support the business logic of having multiple deliveries to complete an order fulfilment. This design is to accommodate the nature of having to pick up multiple items from warehouses and finally deliver it to the customer
 - Each request form only allows for 1 delivery instance to be created with type "TO_CUSTOMER" indicating the delivery of the final complete packages to the customer. It allows for multiple "WAREHOUSE_PICKUP" type of deliveries to be created in a batch and all deliveries having the same signature which is order id, indicating that these deliveries are to fulfil said order.
 - The design also considers to preserve the data integrity of "TO_CUSTOMER" deliveries. In detail, the system will not allow the delivery with "TO_CUSTOMER" to have another status from "PLACED" when "WAREHOUSE_PICKUP" deliveries with the same order id haven't been marked as "DELIVERED".
 - /{deliveryId} : get all information of that delivery
 - /orderId/{orderId}: get all deliveries information associated with that order
 - The design satisfied the requirement of DeliveryCo by letting Store application send its delivery requests, it allowed Store to create many (if applicable) delivery instances for warehouse pickups and a final delivery to the customer. Additionally, it provides QOL features by letting the Store retrieve all details of a delivery associated with a customer order, this allows for real-time tracking of the logistics status.
 - DeliveryCo saves and sets up a scheduled task to update the delivery status after every 1 minute and has a 10% chance to change the delivery status to LOST from any initial state(except for "PLACED" and "DELIVERED", simulate an incident of losing a package).
 - Finally, DeliveryCo implements **message queue feature** to send updates on delivery status to Store app for realtime resource state observation. DeliveryCo also implements an event listener over its message queue channel for incoming messages from BankApp to trigger delivery initiation when an order has been paid.

4. Email Service:

1. **Database:** Keep tracks of emails, log of email sending operations, email metadata and recipient's information as relations: EmailContent, EmailLog, EmailTemplate, Recipient. The important resource's state indicator attributes are being standardized by abstraction into enum types.
2. **Application System:** System implements the MVC design pattern standardised by Spring Boot. Having model, repository, service and controller layers handle and fulfilment functionalities and business logic. For EmailService, the services layer is the heaviest because of the traffic coming into this application (from Store on multiple events from DeliveryCo and BankApp). With each

different type of events (order updates, delivery fulfilment updates, payment status updates) EmailService used a dedicated template to facilitate standardised data objects throughout its operations.

- **Restful APIs:** RestAPIs are responsible for propagating email sending events from Store apps, this unit supports POST and GET request with separation of concerns on different resources (order, payment, delivery)
- **RabbitMQ:** MQ was implemented to allow for incoming command queues from external applications that subscribes to its queue.

Analysis

1. How are the availability and reliability quality attributes improved in your solution? You need to show what will happen under various partial system failures (failure of one or more application components).

- **Reliability:**

- In every application (Store, DeliveryCo, EmailService and BankApp) their service layer all utilise @Transactional decorator. This indicates that atomic writes and reads are used, providing a concrete concurrency safe environment when interacting with the system's resources.
- In the BankApp, the propagation of IdempotencyKey ensures atomic updates of transactions and lets the app validate whether the latest update links to an actual valid request. This removes the possibility of double writes of refund/payment ensuring the reliability of the application.
- In the DeliveryCo, strict business rules are applied
 1. Only allow for a single "TO_CUSTOMER" type of delivery in each delivery instance .
 2. Enforced state freeze for "TO_CUSTOMER" delivery instance of an order until every "WAREHOUSE_PICKUP" delivery instances' states has reached "DELIVERED" or "LOST".
 3. This ensures that all resources of DeliveryCo keep its integrity and correct propagation of the resource's state into 3rd parties' applications.
- Call-backs via webhooks for DeliveryCo and BankApp ensures that any resources' state updates in Store applications are propagated from a valid request from a 3rd party application. Furthermore, asynchronous reads and writes to get these state updates and also a delay in message queues that these updates are traversing through Store application's ecosystem guaranteed that - one, all write and reads are concurrency safe and following thread locking and release of resources state - two, message arrived in the message queues are correctly cached and subsequently processed in first come first serve manner. Hence, due to this design choice, availability and reliability is being greatly prioritised.
- gRPC are implemented for warehouse and inventory management, because of its low latency, strong typing; bi-directional streams later; easy idempotency & timeouts. This service is configured with strict respect to concurrency safe configurations via appropriate concurrency safe function calls when reading or writing on a resource.
- **Failure scenarios:**
 1. Another client triggers an operation on a resource from the warehouse that is being processed by another client: Following first comes first serve, the client that initiate the first request has the involved resource in the "reserve" process, this "reserve" are an abstraction from the client initial order (like how many items they are getting). Therefore if the later client reaches the system, this client will see the inventory's count subtracted with the correct amount in the "reserve" of the previous client. Until the first client goes through with their order or cancels their order, then the termination of this client's "reserve" will release the amount back to the resources of the warehouse system.

2. Client sends an invalid request to the warehouse, the system detects invalid request from the client by using its already concurrency safe state preservation to validate the incorrectness of client request. Therefore, even if the invalid request goes through the initial operations and reaches the point of actual calculation in the repository layers, possible roll-back on failure will kick in to preserve the system integrity and release the faulty request.
3. Client tries to cancel a payment, then tries to reload the client and send a pay request to Store again. In this scenario, the Store application only writes the first states of the payment (FAILED) and any other attempts using client refresh to update the states of the already failed payment have no effect on its state. This ensures integrity and reliability.

○ **Availability:**

- All applications communicate under the standardized response scheme for HTTP and message queue using dedicated templates. This lowers the chance of exception raises and ensures availability.
- Call-backs via webhooks for DeliveryCo and BankApp ensures that any resources' state updates in Store applications are propagated from a valid request from a 3rd party application. Furthermore, asynchronous reads and writes to get these state updates and also a delay in message queues that these updates are traversing through Store application's ecosystem guaranteed that - one, all write and reads are concurrency safe and following thread locking and release of resources state - two, message arrived in the message queues are correctly cached and subsequently processed in first come first serve manner. Hence, due to this design choice, availability and reliability is being greatly prioritised.
- External HTTP requests coming from webhooks have strict timeouts to avoid blocking other incoming requests. Any request missing the timeouts limit will be rejected by the system and the system will terminate its timeout state to continue on listening for other requests. However, to further increase availability, the system implementation allows for timeout-ed requests to wait for retries or dead-letter queues, allowing the core web path to stay responsive.

■ **Failure Scenario:**

1. BankApp timeout or temporary out of service:
 - Initial charge times out, then publish "check payment status" message
 - Client retries with 3/6/9 seconds wait time.
 - After all allowed repeated failure, Store application route to DLQ and persist record.
 - User gets a clear error log of code 500 and no state changes will be made on their resources
2. DeliveryCo timeout or temporary out of service:
 - DeliveryCo return error code 500
 - Store apps trigger the procedure of rolling back all its states to initial states after receiving the 500 error code.

3. Warehouse gRPC out of service:

- Inventory and warehouse state will be frozen in the state before the failure happens.
- All incoming requests will be rejected with appropriate error code to let clients know the service is unavailable.
- Store applications transactions service perform a roll-back to avoid inconsistency.
- The store sends a response back to the user with the error code, encouraging them to retry.

2. Apart from availability and reliability, how well are the other quality attributes mentioned in the first lecture incorporated by your design? Consider and discuss trade-offs between different related quality attributes.

- Aside from reliability and availability, the other “ibilities” that the system displayed are maintainability, scalability but the trades off here are performance and modifiability.
- The system displays a clear separation of concerns and atomic units and layers of services, making maintenance tasks to the application code more modular and straight-forward.
- Scalability is another strong point of this implementation because of its modularity and the amount of abstraction it has. Each module can scale vertically to increase the overall capability of the system, hence increasing the maximum possible load that the system can handle.
- However, performance is one of the trade-off of this implementation because of front-loading hefty amount of communication procedures for a single chain of operations (i.e: to make an order, the system has to handle internal operations of creating the order, then negotiating resources and states with other 3rd party applications using time gated webhooks and message queues with retries and dead -letter enables). These operations ensure correctness and availability but drag out the time for the system to fulfil its functionality. This will become more noticeable if the 3rd party applications don't handle webhooks and message queues integration well with the Store app.
- Modifiability is also a trade-off because of how each module functionally depends on each other. Here we take a small and atomic operation as an example, if we want to add another feature into warehouse module, we would need to also first integrate into the gRPC flow and adhere to the current concurrency safe design pattern then apply that change into the resources that the warehouse will be handling and create another adapter in the Store app to accommodate this feature if its scope involve the Store apps,... This implementation is not tightly coupled in nature but making changes in one module leads to changes in many related modules making it difficult to modify any modules in this implementation.
- The most important thing in this game of trades off is to find the sweet-spot for the final composition of “ibilities” to have in our final products. Here in this implementation we aimed to promote reliability and availability to the best possible hence the lack of other characteristics.

3. How does the use of an ORM database interface model affect the implementation of the tiered architecture within your Store application? Does your design maintain a clear separation between layers, or are there any areas where the architecture becomes blurred? Plotting the ERD Diagram should help.

- With the use of an ORM database interface model, repository layers of all the applications are able to isolate SQL details and reduce the actual resources exposure to other modules in the system. The use of an ORM also facilitates independence of the application system from the database, specifically the application can freely impose business rules and logic on top of the repository caller instead of tailoring coupled logic and business rules together to accommodate the data layer.
- With the presence of the ORM, the architecture overall becomes more modular and easier to understand with separation of concern clearly drawn at the repository layer. Other layers such as service and controllers also have clear separation with each other, it can be seen with the implementation of each module's preference of having association relationship rather than composition.
- However, some modules and its related operations have blurred separation of concerns such as seen in the Store application where inventory management happens. In the service layer concerning management of Inventory, external HTTP handlers are mixed with gRPC and messaging operations, or in the BankApp - AccountService, where declaration of data types is made amidst of management operations of Account resources.